

TP6 - SVM

Reprenons l'exemple du cours : $\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 2 & 2 \\ 2 & 0 \\ 3 & 0 \end{bmatrix}$ et $y = \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix}$

I. Plan séparateur

Utilisez la fonction `aff_donnees(X, y, bornex, borney, s)` (display) qui affiche les données sur une figure en mettant un symbole différent pour les exemples positifs et négatifs. `bornex`, `borney` représentent les limites des axes que l'on pourra fixer à min-1 et max+1. `s` représente la taille des symboles (utile dans la dernière partie du TP) que l'on pourra fixer à 50.

```
bornex=[min(X[:,0])-1,max(X[:,0])+1]
borney=[min(X[:,1])-1,max(X[:,1])+1]
```

Ecrivez une fonction `affichePlan(w, b, bornex)` qui affiche un hyperplan dans le plan 2D en passant en argument les paramètres de l'hyperplan $\mathbf{w}$ et b ainsi que `bornex`. On pourra pour cela faire varier x dans l'intervalle défini par `bornex` et calculer les y correspondants. Affichez l'hyperplan de paramètre $\mathbf{w}^T = [1, 0.1]$ et $b = -1$.

Questions

S'agit-il d'un hyperplan séparateur ?

II. SVM linéaire dans le primal

On a vu en cours que le problème des SVM peut être mis sous la forme d'un problème quadratique dont la solution est obtenue avec la librairie `cvxopt`. Complétez la fonction `ResoudPrimal(X, y)` qui renvoie les paramètres $\mathbf{w}$ et b de l'hyperplan séparateur.

Rq : la fonction `sol = cvxopt.solvers.qp(P, q, G, h)` résout le problème :

$$\min_z 0.5z^T Pz + q^T z \quad \text{sc} \quad Gz \leq h$$

P, q, G, h doivent être des matrices décimales de type `cvxopt.matrix`. La conversion est réalisée avec `P= cvxopt.matrix(P)`. La solution au problème est trouvée avec `z=sol['x']`.

Complétez pour cela la fonction `Resoud_primal(X, y)` (fn)

Vérifiez votre programme en traçant cet hyperplan.

Questions

Est-ce que l'hyperplan obtenu vous paraît correct ?

Que se passe-t-il si on ajoute le point $\mathbf{x} = \begin{bmatrix} 2.1 \\ 2.5 \end{bmatrix}$ d'étiquette 1 ? Vérifiez en faisant tourner le programme.

Même question avec le point $\mathbf{x} = \begin{bmatrix} 1.5 \\ 2.5 \end{bmatrix}$

III. SVM à marge souple

On va utiliser l'apprentissage SVM de python qui autorise les marges souples.

```
model = svm.SVC(kernel='linear', C=???)  
model.fit(X, y)  
w = model.coef_[0]  
b = model.intercept_[0]
```

Affichez l'hyperplan obtenu avec la fonction `aff_plan()` précédente puis avec la fonction `aff_frontiere()` donnée en annexe. Que réalise la fonction `aff_frontiere()` ?

Questions

Que représente la variable C

Testez le programme sur les données initiales. Conclusion ?

Ajoutez le point $\mathbf{x} = \begin{bmatrix} 2.1 \\ 2.5 \end{bmatrix}$ d'étiquette 1 et testez plusieurs valeurs de C. Conclusion ?

Ajoutez le point $\mathbf{x} = \begin{bmatrix} 1.5 \\ 2.5 \end{bmatrix}$ d'étiquette 1 et testez plusieurs valeurs de C. Conclusion ?

IV. SVM avec Kernel

Dans cette partie, on travaillera sur les données «TP6.npz » :

```
f = np.load('TP6.npz',allow_pickle=True)  
X=f['arr_0']  
y=f['arr_1']
```

Apprenez directement un SVM avec un kernel linéaire et affichez la fonction de décision avec `aff_frontiere(X,y,bornex,borney,model) (display)`.

Apprenez des SVM avec des noyaux linéaire, rbf et polynomial. Que remarquez-vous sur les nouvelles fonctions de décisions ?

Questions

Peut-on apprendre des frontières non linéaires avec un noyau linéaire ?

Que donne un SVM avec noyau rbf sur les données linéairement séparables (données de départ) ? Est-il bien adapté ?